EXPERIMENT 7

Academic Year: 2026-27 Year and Sem: TE V
Subject: DEV Lab Course Code: 2175119

PROBLEM STATEMENT:

To deploy and manage containerized applications using Kubernetes.

Requirements:

Ubuntu 22.04 , Docker Installed , Minikube Installed , kubectl Installed

THEORY:

Container Orchestration: Introduce the concept of container orchestration and its importance in managing containerized applications at scale. Discuss challenges such as service discovery, load balancing, scaling, and automated rollouts.

Kubernetes: Explain what Kubernetes is and how it addresses the challenges of container orchestration. Cover key Kubernetes concepts, including Pods, Deployments, Services, ReplicaSets, and ConfigMaps.

Kubernetes Architecture: Describe the architecture of Kubernetes, including the control plane components (e.g., API server, scheduler, controller manager) and worker node components (e.g., kubelet, kube-proxy, container runtime).

Kubernetes Objects: Provide an overview of Kubernetes objects and their YAML or JSON representations. Explain how to define Pods, Deployments, Services, ConfigMaps, and other

resources using Kubernetes manifests.

Kubernetes Networking: Discuss Kubernetes networking concepts, including cluster networking, Pod networking, and Services. Explain how Kubernetes manages network communication between Pods and Services.

Implementation Steps:

Introduction to Kubernetes: Present an overview of Kubernetes technology and its role in container orchestration.

Installing Kubernetes: Guide participants through the process of installing a Kubernetes cluster using Minikube for local development or kubeadm for production-like clusters. 3. **Kubernetes Basics**: Demonstrate basic Kubernetes commands using kubectl for cluster administration, such as creating Pods, Deployments, and Services.

Deploying Applications: Instruct participants to deploy sample applications (e.g., web server, database) using Kubernetes Deployments and Services.

Scaling Applications: Show participants how to scale applications dynamically based on resource demands using Kubernetes Horizontal Pod Autoscaler (HPA).

Updating Applications: Walk participants through the process of performing rolling updates to application deployments with Kubernetes, ensuring zero downtime. 7. **Managing Configuration**: Discuss best practices for managing application configurations using Kubernetes ConfigMaps and Secrets.

Monitoring and Logging: Introduce participants to Kubernetes monitoring and logging solutions, such as Prometheus for monitoring and Fluentd for logging

Installation of Minikube and kubectl on Ubuntu

Step 1: Update System Command:

sudo apt update

sudo apt upgrade -y

Step 2: Install Required Dependencies

sudo apt install curl wget apt-transport-https -y

Step 3: Install Docker (required for Minikube)

sudo apt install docker.io -y

sudo systemctl start docker

sudo systemctl enable docker

sudo usermod -aG docker

$USER newgrp docker

docker --version

Step 5: Install Minikube Commands:

curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube-linux-amd64 /usr/local/bin/minikube

minikube version

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ curl -LO https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linux/amd64/kubectl
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 53.7M  100 53.7M    0     0  8736k      0  0:00:06  0:00:06 --:--:-- 11.5M
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ chmod +x kubectl
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ sudo mv kubectl /usr/local/bin/
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ kubectl version --client
Client Version: v1.31.0
Kustomize Version: v5.4.2
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  128M  100  128M    0     0  11.6M      0  0:00:11  0:00:11 --:--:-- 16.5M
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
```

Step 6: Start Minikube Cluster Command:

minikube start --driver=docker

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ minikube start --driver=docker
😄  minikube v1.38.1 on Ubuntu 22.04
✨  Using the docker driver based on user configuration
❗  Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
📌  Using Docker driver with root privileges
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.50 ...
💾  Downloading Kubernetes v1.35.1 preload ...
    > preloaded-images-k8s-v18-v1...:  272.45 MiB / 272.45 MiB  100.00% 14.23 M
    > gcr.io/k8s-minikube/kicbase...:  519.58 MiB / 519.58 MiB  100.00% 15.35 M
🔥  Creating docker container (CPUs=2, Memory=3072MB) .../E0320 15:15:07.105743    6810 network_create.go:104] failed to find free subnet for docker network minikube after 20 attempts: no free private network subnets found with given parameters (start: "192.168.49.0", step: 9, tries: 20)
🔥  Creating docker container (CPUs=2, Memory=3072MB) ...
❗  Unable to create dedicated network, this might result in cluster IP change after restart: un-retryable: no free private network subnets found with given parameters (start: "192.168.49.0", step: 9, tries: 20)
🐳  Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...|E0320 15:15:22.361590    6810 start.go:134] Unable to get host IP: inspect gateway: inspect gateway: docker container inspect --format {{.NetworkSettings.Gateway}} minikube: exit status 1
stdout:


stderr:
template parsing error: template: :1:18: executing "" at <.NetworkSettings.Gateway>: map has no entry for key "Gateway"
🐳  Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
❗  Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine
💡  To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
🔗  Configuring bridge CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass

❗  /usr/local/bin/kubectl is version 1.31.0, which may have incompatibilities with Kubernetes 1.35.1.
    ▪ Want kubectl v1.35.1? Try 'minikube kubectl -- get pods -A'
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Step 7: Verify Installation Commands:

kubectl cluster-info kubectl get nodes

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ kubectl cluster-info
kubectl get nodes
Kubernetes control plane is running at https://172.17.0.2:8443
CoreDNS is running at https://172.17.0.2:8443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   63s   v1.35.1
```

Step 8: Test Kubernetes Deployment Commands:

kubectl create deployment nginx --image=nginx

kubectl get pods

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ kubectl get pods
NAME                     READY   STATUS              RESTARTS   AGE
nginx-56c45fd5ff-tsz2h   0/1     ContainerCreating   0          5s
```

Step 9: Expose the Application

Command:

kubectl expose deployment nginx --type=NodePort –port=80

kubectl get services

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ kubectl expose deployment nginx --type=NodePort --port=80
kubectl get services
service/nginx exposed
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1      <none>        443/TCP        3m38s
nginx        NodePort    10.111.7.205   <none>        80:30422/TCP   0s
```

Step 10: Access the Application

minikube service nginx

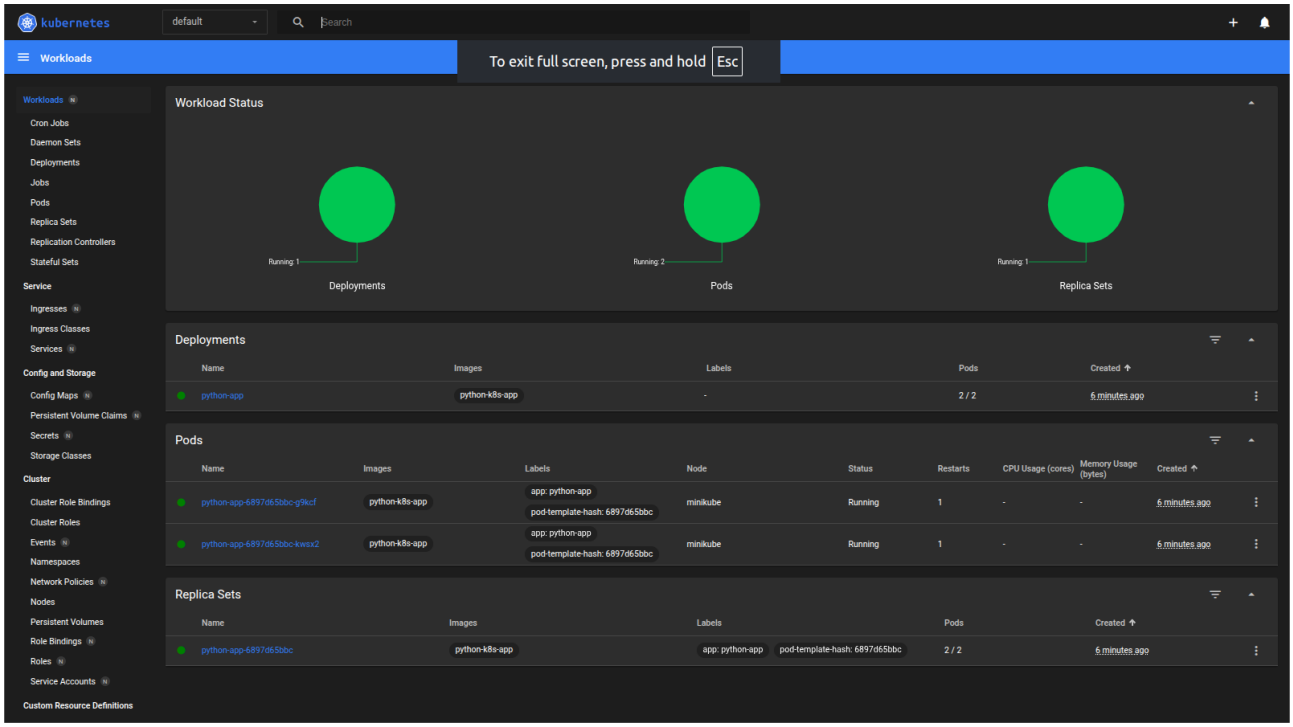
```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ minikube service nginx
```

NAMESPACE	NAME	TARGET PORT	URL
default	nginx	80	http://172.17.0.2:30422

```
🎉  Opening service default/nginx in default browser...
```

Step 11: Open Kubernetes Dashboard

Command: minikube dashboard



Step 12: Scale the Application

kubectl scale deployment nginx –replicas=4

kubectl get pods

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ kubectl scale deployment nginx --replicas=4
kubectl get pods
deployment.apps/nginx scaled
NAME                     READY   STATUS              RESTARTS   AGE
nginx-56c45fd5ff-7bd49   0/1     ContainerCreating   0          0s
nginx-56c45fd5ff-tsz2h   1/1     Running             0          3m50s
nginx-56c45fd5ff-vvf6t   0/1     Pending             0          0s
nginx-56c45fd5ff-w59c8   0/1     ContainerCreating   0          0s
```

Step 13: Delete Resources

Command:

kubectl delete deployment nginx

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ kubectl delete deployment nginx
minikube stop
minikube delete
deployment.apps "nginx" deleted
✋  Stopping node "minikube"  ...
🛑  Powering off "minikube" via SSH ...
🛑  1 node stopped.
🔥  Deleting "minikube" in docker ...
🔥  Deleting container "minikube" ...
🔥  Removing /home/apsit/.minikube/machines/minikube ...
💀  Removed all traces of the "minikube" cluster.
```

Stop Cluster: minikube stop
Delete Cluster: minikube delete

Python Application Deployment in Kubernetes (Minikube + Docker)
Deploying a Python Application in Kubernetes using Minikube and Docker (Ubuntu)
Step 1: Create a Project Folder
Open the terminal and run:
mkdir python-k8s
cd python-k8s

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~$ mkdir python-k8s
cd python-k8s
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/python-k8s$
```

Step 2: Create Python Application
Create the file:
nano app.py
Paste the following code:

from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
return "Hello from Python App running in Kubernetes!"
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)

```
  GNU nano 6.2                          app.py *
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Python App running in Kubernetes!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Save the file:
CTRL + X

Y
ENTER

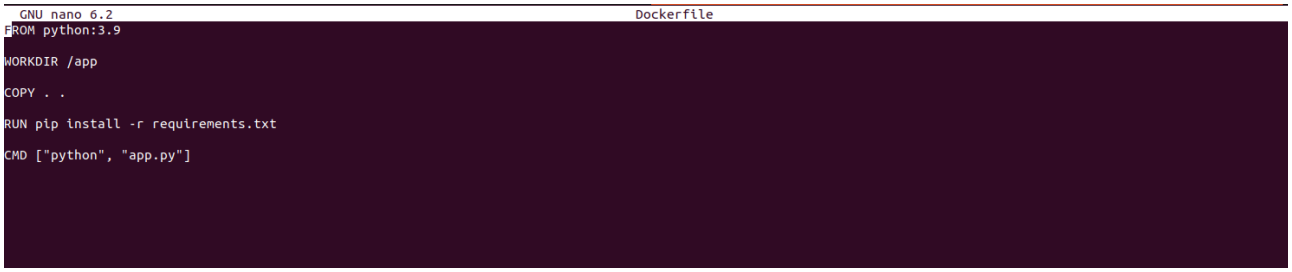
Step 3: Create Requirements File
Create the file:
nano requirements.txt

Add the following line:
flask



Save and exit.
Step 4: Create Dockerfile
Create the file:
nano Dockerfile
Paste the following:

FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
Save and exit.



Step 5: Use Minikube Docker Environment

Run the following command:

eval $(minikube docker-env)

This allows Docker images to be built inside Minikube.



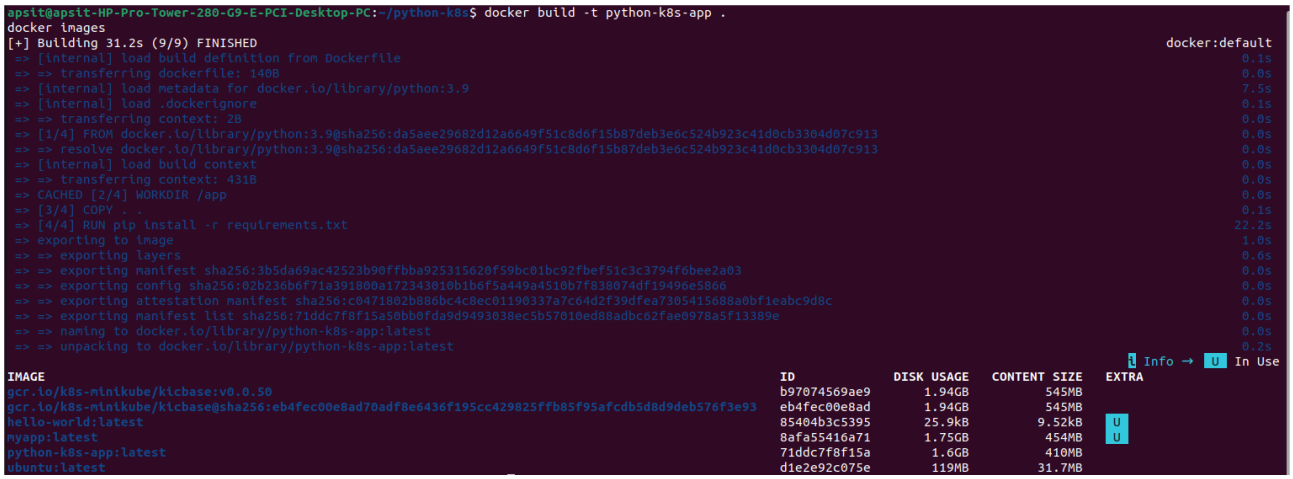
Step 6: Build Docker Image

Run:

docker build -t python-k8s-app .

Check the image:

docker images



Step 7: Create Kubernetes Deployment File

Create the file:

nano deployment.yaml

Paste the following configuration:

apiVersion: apps/v1

kind: Deployment

metadata:

name: python-app

spec:

replicas: 2

selector:

matchLabels:

app: python-app

template:
metadata:
labels:
app: python-app
spec:
containers:
- name: python-app
image: python-k8s-app
imagePullPolicy: Never
ports:
- containerPort: 5000
Save and exit.

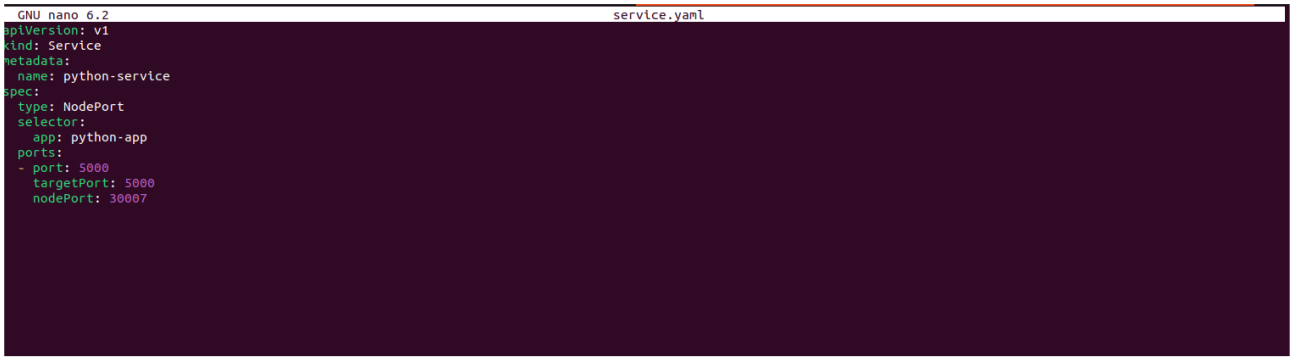
```
 GNU nano 6.2                                                deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: python-app
  template:
    metadata:
      labels:
        app: python-app
    spec:
      containers:
      - name: python-app
        image: python-k8s-app
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
```

Step 8: Deploy Application to Kubernetes
Run:
kubectl apply -f deployment.yaml
Check deployment:
kubectl get deployments
Check pods:
kubectl get pods
You should see two running pods.

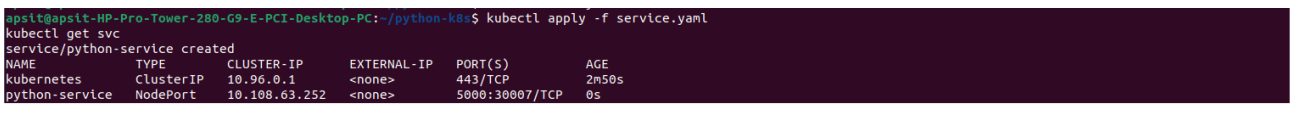
```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/python-k8s$ kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get pods
deployment.apps/python-app created
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
python-app   0/2     2            0           0s
NAME                          READY   STATUS              RESTARTS   AGE
python-app-6897d65bbc-4nnkz   0/1     ContainerCreating   0          0s
python-app-6897d65bbc-zpcpc   0/1     ContainerCreating   0          0s
```

Step 9: Create Service File
Create the file:
nano service.yaml
Paste the following:
apiVersion: v1
kind: Service

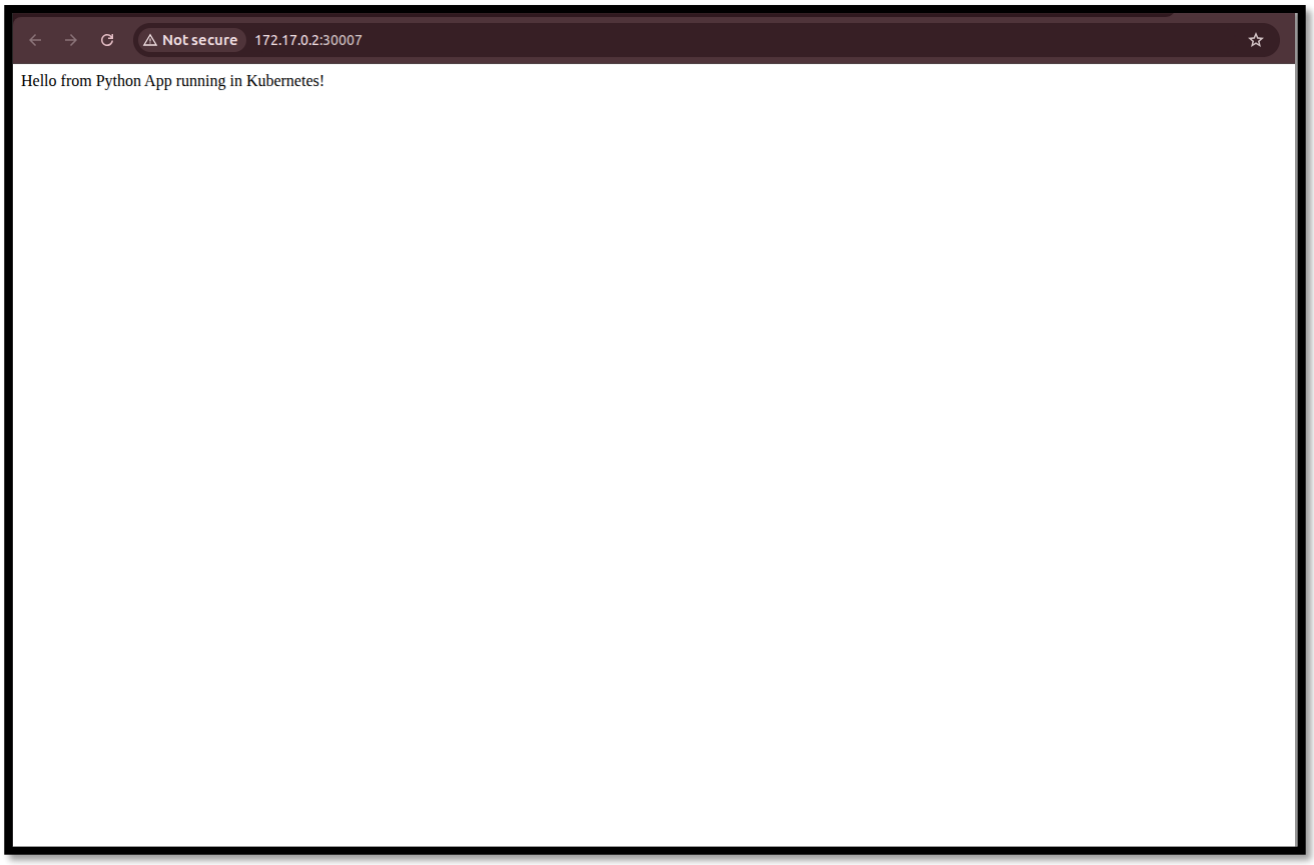
metadata:
name: python-service
spec:
type: NodePort
selector:
app: python-app
ports:
- port: 5000
targetPort: 5000
nodePort: 30007
Save and exit.



Step 10: Expose the Service
Run:
kubectl apply -f service.yaml
Check the service:
kubectl get svc



Step 11: Open the Application
Run:
minikube service python-service
Your browser will open and show:
Hello from Python App running in Kubernetes!



Step 12: Verify Everything

Run the following commands:

kubectl get deployments

kubectl get pods

kubectl get svc

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/python-k8s$ kubectl get deployments
kubectl get pods
kubectl get svc
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
python-app   2/2     2            2           2m23s
NAME                          READY   STATUS    RESTARTS   AGE
python-app-6897d65bbc-g9kcf   1/1     Running   0          2m23s
python-app-6897d65bbc-kwsx2   1/1     Running   0          2m23s
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes       ClusterIP   10.96.0.1       <none>        443/TCP          10m
python-service   NodePort    10.108.63.252   <none>        5000:30007/TCP   7m37s
```

Conclusion :

The experiment demonstrated the deployment and management of a containerized application using Kubernetes on a single-node Minikube cluster. Students learned how to create deployments, expose applications using services, monitor pods, scale applications, and delete Kubernetes resources, providing a foundation for managing containerized workloads in cloud-native environments.